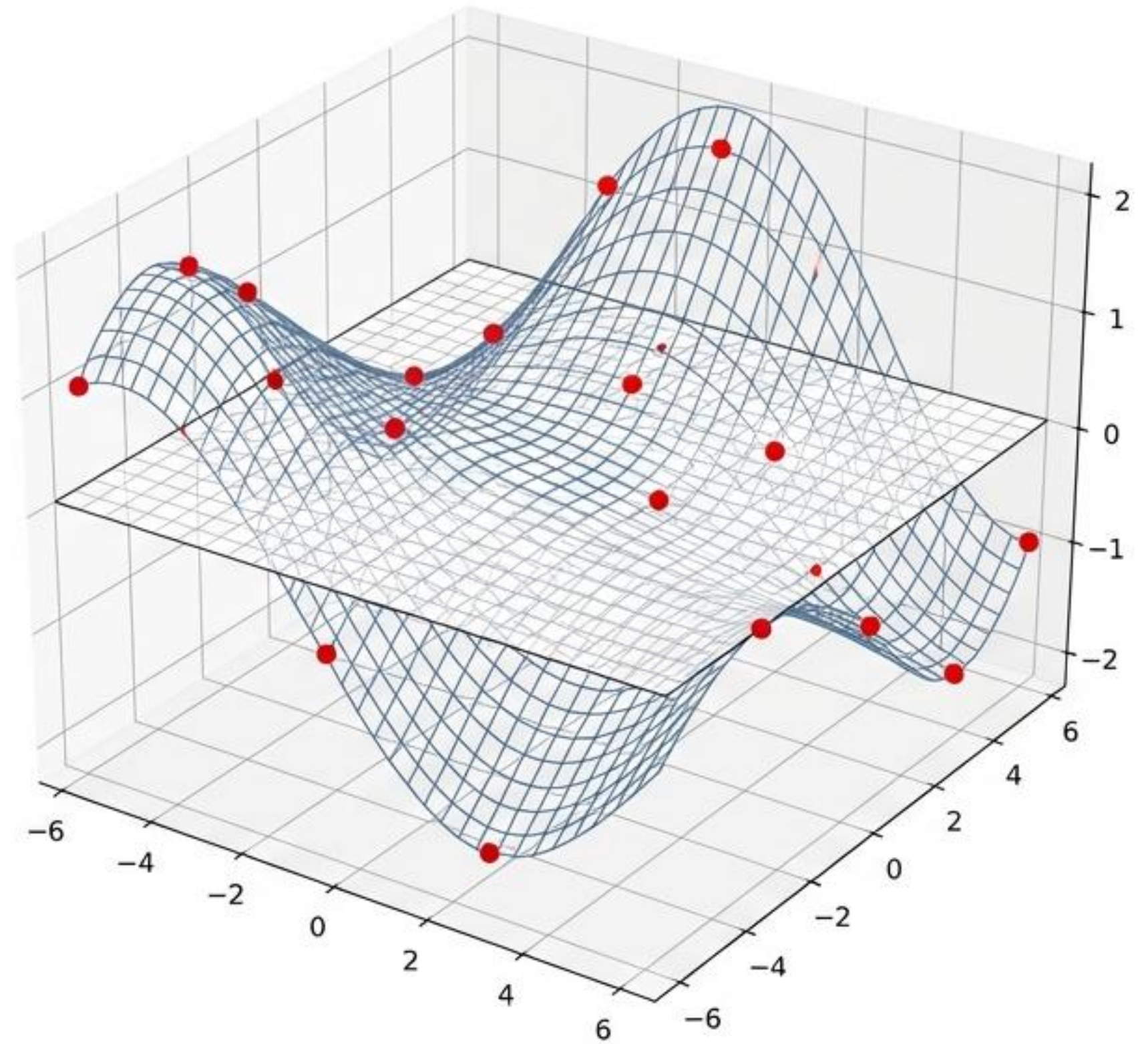


Interpolación Polinomial

Fundamentos Teóricos,
Algoritmos y Aplicaciones en
Ingeniería

Curso: Métodos Numéricos

Docente: Dr. Hermes Pantoja Carhuavilca



Objetivos de la Sesión

CONSTRUIR

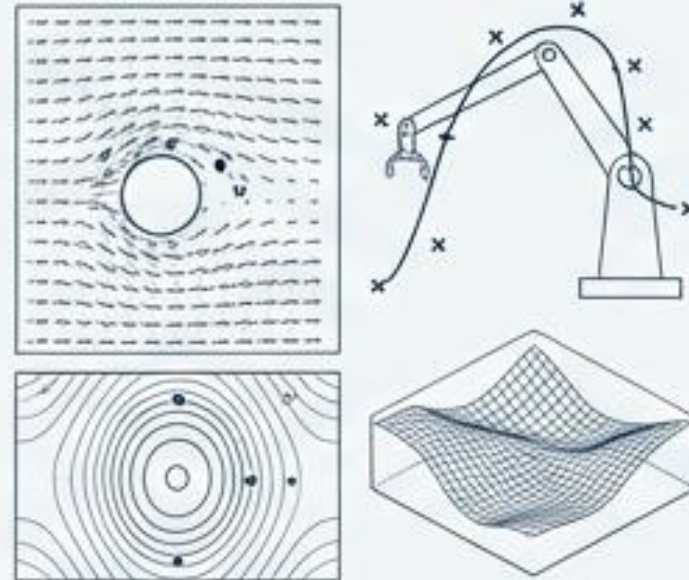
Polinomios interpolantes únicos mediante métodos de Lagrange y Diferencias Divididas (Newton).



$$\begin{aligned} f[x_0, x_1] &\rightarrow x_0 \rightarrow \Delta \\ f[x_1, x_2] &\rightarrow x_1 \rightarrow \Delta \\ f[x_0, x_2] &\rightarrow x_2 \rightarrow \Delta \end{aligned} \rightarrow x_{n+1}$$

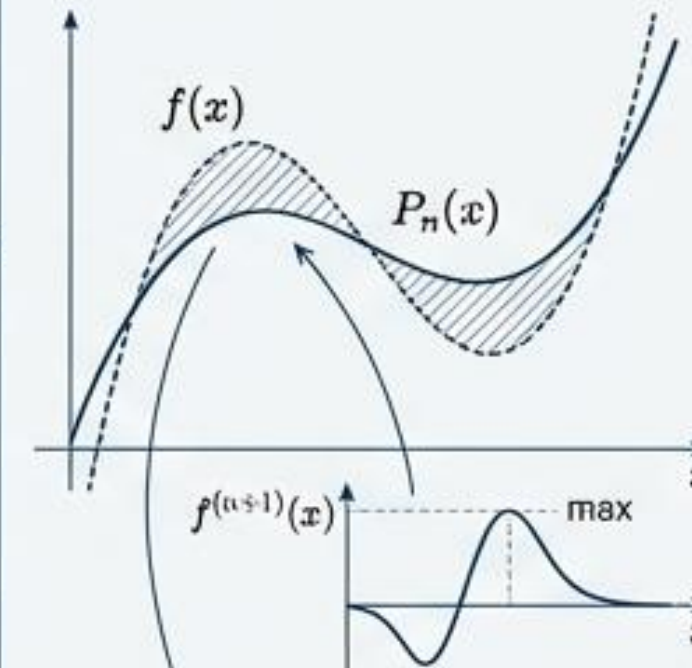
CALCULAR

Estimaciones para datos discretos en fluidos, robótica y topografía donde la experimentación es costosa.



ANALIZAR

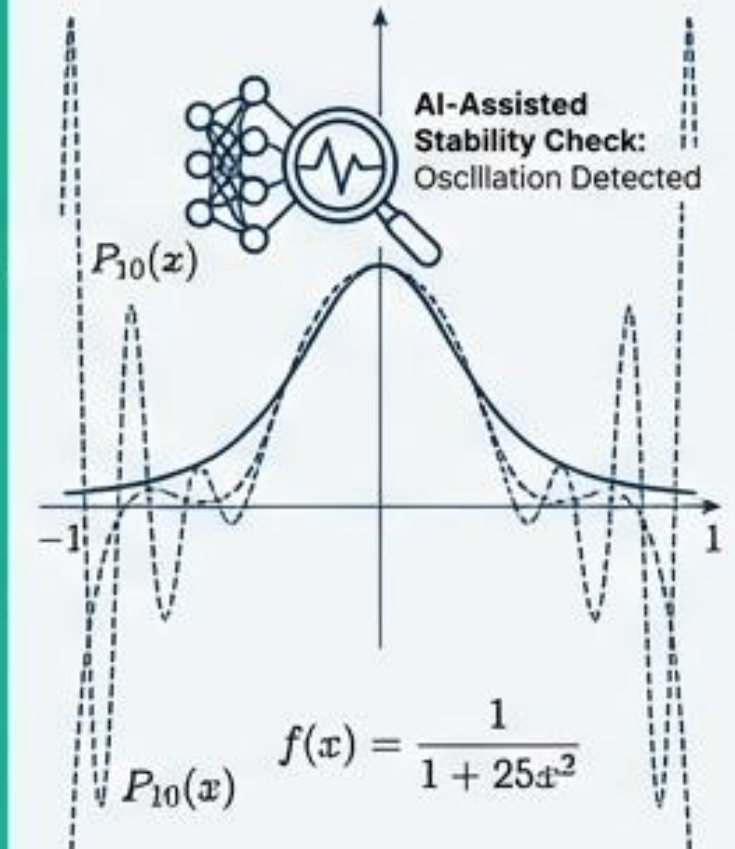
La cota de error E_n , y la influencia de la derivada de orden superior $f^{(n+1)}$.



$$E_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

VALIDAR

Estabilidad numérica frente a oscilaciones (Runge) asistido por IA.



Definición y Alcance

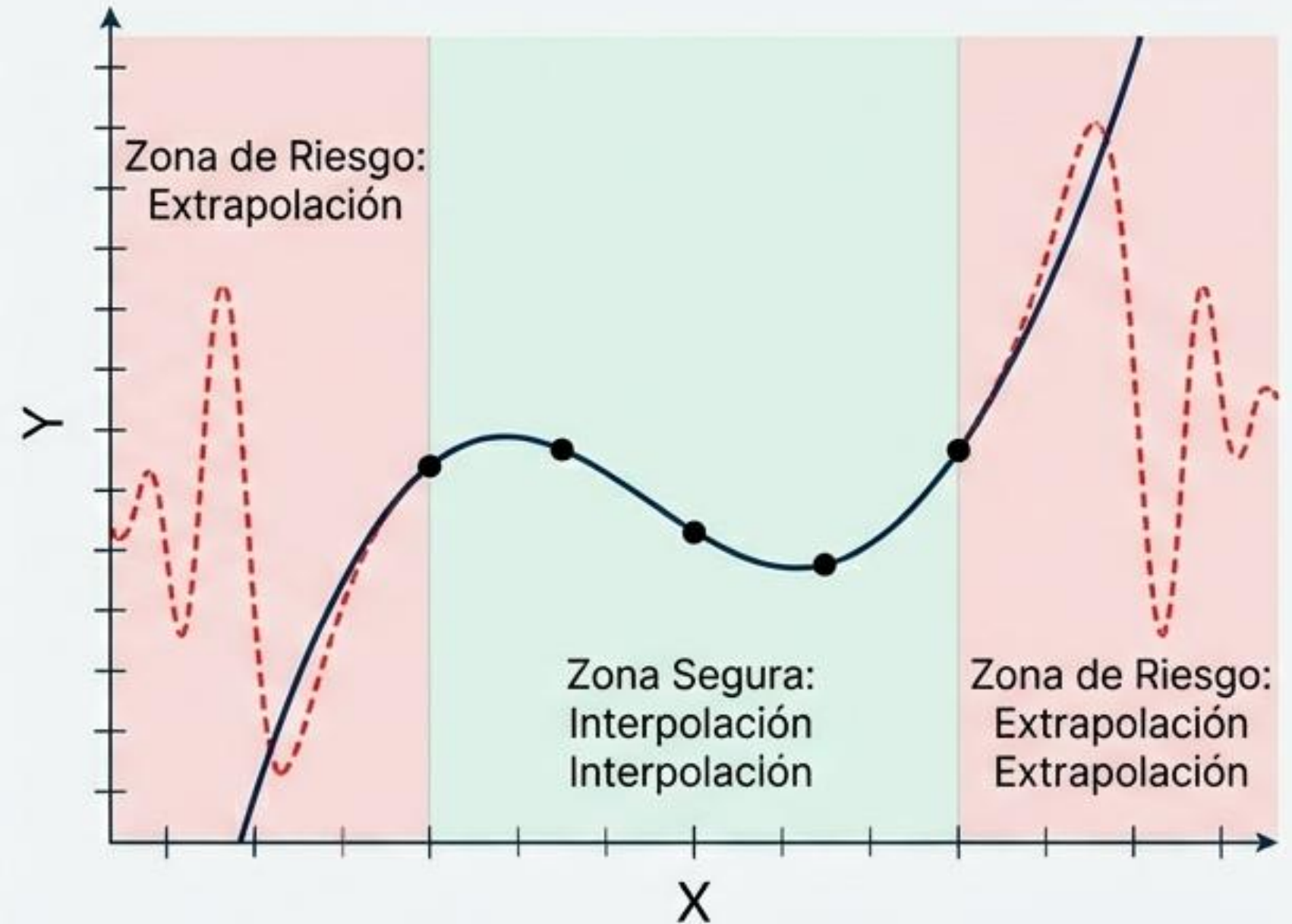
Interpolación vs. Extrapolación

1. Interpolación

- **Definición:** Ajuste de una función $f(x)$ que pasa **exactamente** por puntos discretos conocidos.
- **Objetivo:** Estimar valores intermedios en $[x_0, x_n]$.
- **Fiabilidad:** Alta para funciones suaves.

2. Extrapolación

- **Definición:** Estimación fuera del intervalo conocido.
- **Riesgo:** Altamente volátil. Divergencia de la realidad física.



El ajuste polinomial garantiza el paso por los puntos, pero no garantiza la física fuera de ellos.

Polinomio de Lagrange: Enfoque de Funciones Base

Concepto:

Reformulación del polinomio que evita sistemas de ecuaciones lineales. Se basa en una suma ponderada de términos.

- No requiere resolver matrices.
- Computacionalmente costoso de recalcular si se añaden nuevos puntos.

Fórmula General:

$$P_n(x) = \sum_{i=0}^n y_i L_i(x)$$

Función Base (Producto):

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

$$L_i(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0)(x_i - x_1) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}$$

Polinomio de Newton: Recursividad y Eficiencia

Utiliza Diferencias Divididas Finitas para construir el polinomio incrementalmente. Ideal para programación algorítmica.

$$P_n(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1) + \dots$$

Tabla de Diferencias Divididas

x_i	$f(x_i)$	Primeras Diferencias $f[x_i, x_{i+1}]$	Segundas Diferencias $f[x_i, x_{i+1}, x_{i+2}]$
x_0	$f(x_0)$	$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$ $f[x_1, x_2] = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$	$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$
x_1	$f(x_1)$		
x_2	$f(x_2)$		

Análisis del Error de Interpolación

El error no es cero en los puntos intermedios. Depende de la suavidad de la función (derivadas) y la distribución de los puntos.

Fuente del Peligro: Si la función oscila mucho, este valor explota.

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

Geometría: El error disminuye cerca de los nodos conocidos.

Teorema del Residuo (Definición Formal):

Sea $f(x)$ una función con $n + 1$ derivadas continuas en un intervalo $[a, b]$ que contiene los $n + 1$ puntos de interpolación $x_0, x_1, \dots, x_n$. Entonces, para cualquier $x \in [a, b]$, existe un ξ en el intervalo (que depende de x) tal que el error de interpolación $R_n(x) = f(x) - P_n(x)$ está dado por la fórmula anterior.

Análisis del Error de Interpolación

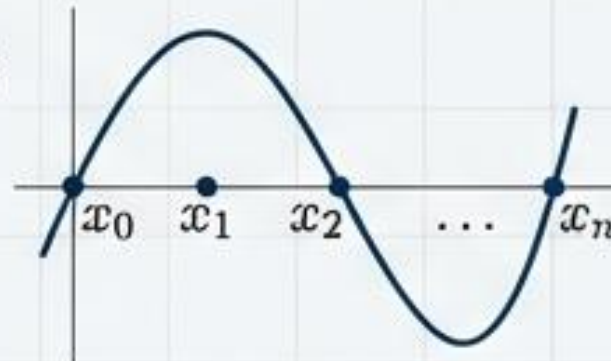
Descomposición de la fuente de error

$$E_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

Donde ξ es un valor desconocido en el intervalo (x_0, x_n) .

Factor Geométrico $\prod (x - x_i)$

El error es cero en los nodos y máximo cerca del punto medio entre ellos.



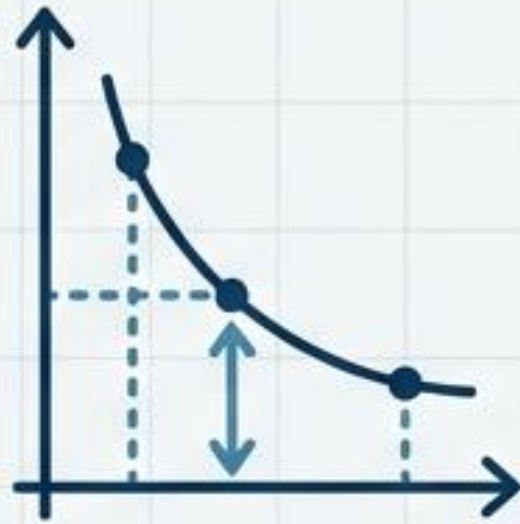
Factor de Suavidad $f^{(n+1)}(\xi)$

Depende de las variaciones de la función. Si la función oscila bruscamente (derivadas altas), el error será masivo.

Mayor grado (n) no siempre reduce el error si la derivada crece rápidamente.

Aplicaciones I: Sistemas Industriales y Mecánicos

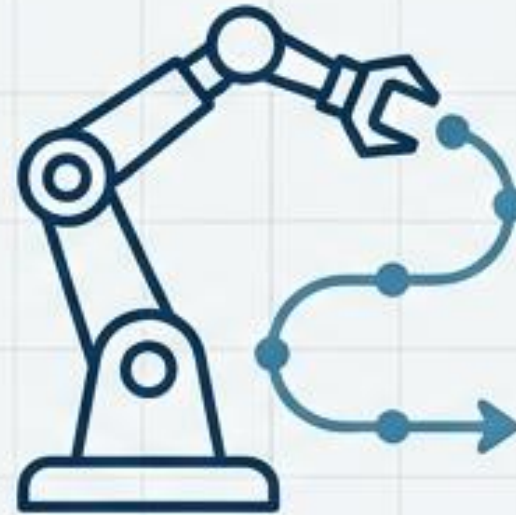
Ingeniería Industrial



Contexto: Curvas de economía de escala no lineales con históricos incompletos.

Aplicación: Interpolar costos unitarios para volúmenes de producción no registrados (ej. 1500 unidades) para presupuestos precisos.

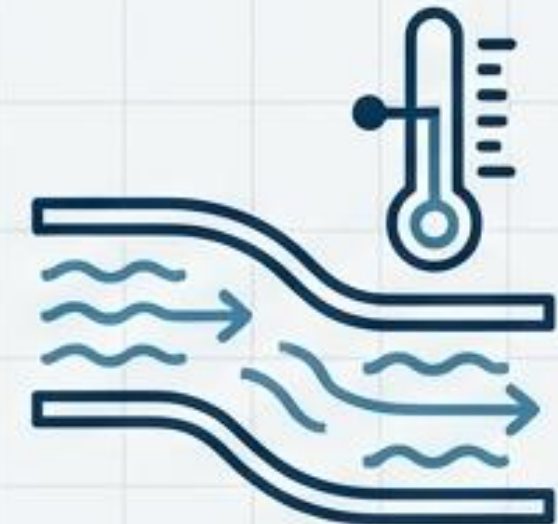
Mecatrónica / Robótica



Contexto: Controladores que reciben coordenadas discretas (waypoints).

Aplicación: Generar trayectorias suaves entre puntos para evitar movimientos bruscos (jerk) y proteger servomotores.

Ingeniería Mecánica (Fluidos)



Contexto: Tablas de vapor/fluidos con datos a grandes saltos de temperatura.

Aplicación: Calcular viscosidad/entalpía exacta a temperatura de operación ($T=123.5^{\circ}\text{C}$) para diseño térmico.

Aplicaciones II: Infraestructura y Energía

Energía (Gestión de Demanda)



Contexto: Pérdida de datos en medidores inteligentes.

Aplicación: Reconstruir curvas de demanda horaria para predecir picos y dimensionar protecciones eléctricas.

Ambiental (Sensores Remotos)



Contexto: Fallo de sensores de contaminación en momentos críticos.

Aplicación: Estimar niveles de NOx/CO2 en tiempos sin muestra, basándose en tendencias adyacentes para cumplimiento normativo.

Civil (Topografía)



Contexto: Levantamientos topográficos (LiDAR) con nubes de puntos.

Aplicación: Generar perfiles continuos de terreno para calcular volúmenes de corte y relleno en carreteras.

Reto Crítico: La Trampa del Fenómeno de Runge

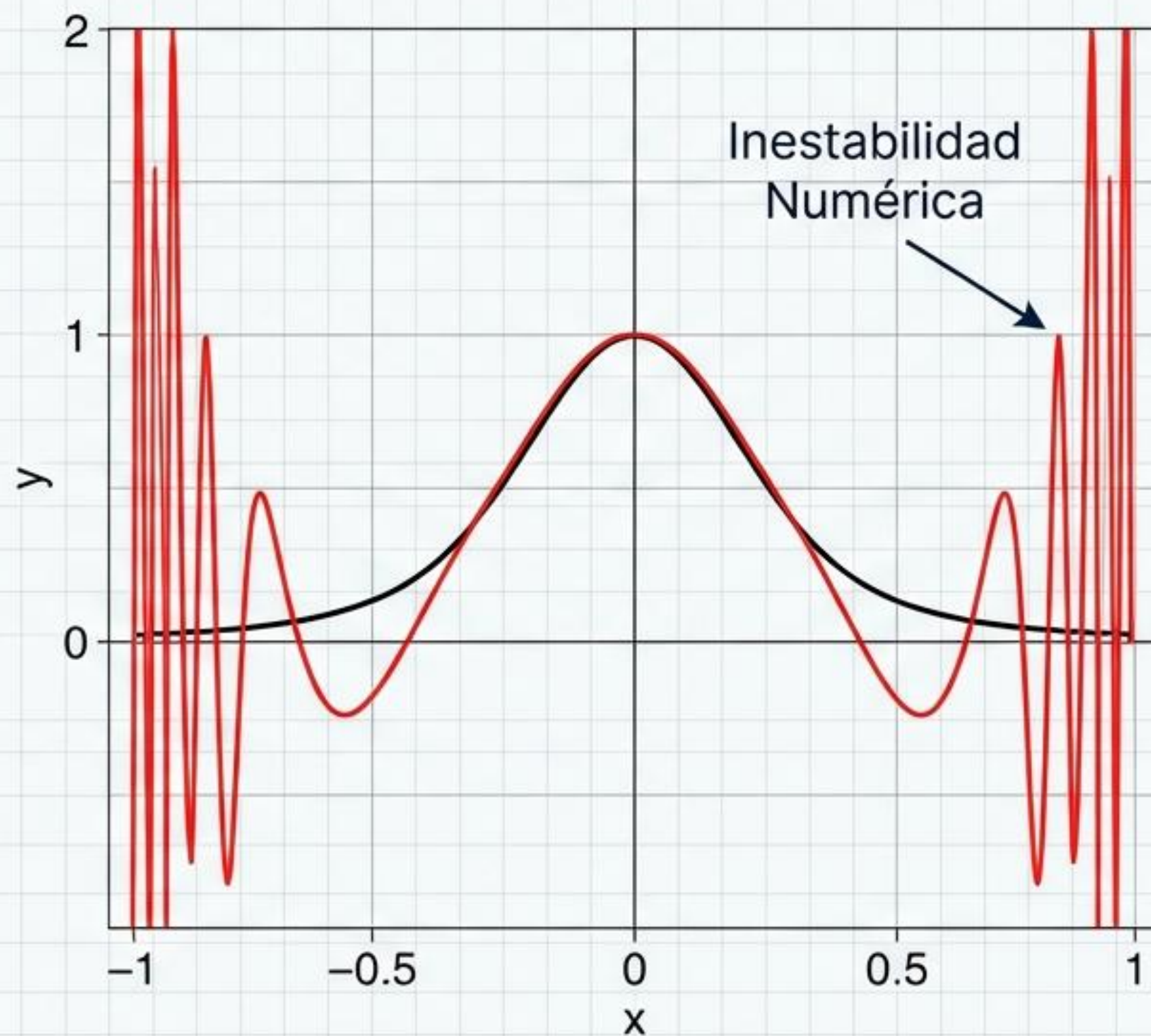
El Problema:

Al interpolar funciones específicas con puntos equidistantes, el error crece exponencialmente al aumentar el grado del polinomio.

Función de Prueba:

$$f(x) = \frac{1}{1 + 25x^2} \text{ en } [-1, 1]$$

Hecho Contraintuitivo: Más datos no siempre significan mejor aproximación.



Reto IA: Generación y Simulación

Terminal / Editor de Código

```
1  **Prompt sugerido para IA:**  
2  
3  > "Genera un script en MATLAB/Python para interpolar la  
4  función de Runge  $f(x) = 1/(1+25x^2)$  en el intervalo  $[-1, 1]$   
5  usando un polinomio de Newton con 15 puntos equidistantes.  
6  Grafica la función real vs. la interpolación."
```

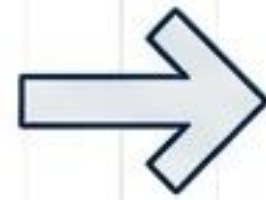
Objetivo del alumno:

Ejecutar el código y observar visualmente la **divergencia** en los extremos $x \approx \pm 1$.

Análisis Crítico: ¿Por qué falla el modelo?

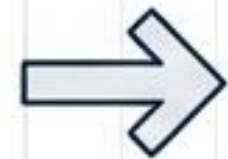
1. Culpable: Derivadas de Alto Orden.

En la función de Runge, las derivadas superiores crecen factorialmente.



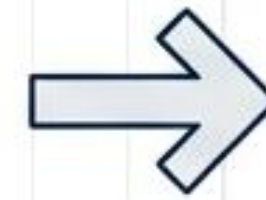
2. Mecanismo: El Término de Error.

$$Error = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$



El crecimiento de $f^{(n+1)}(\xi)$ supera la reducción del producto de distancias.

$$Error = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$



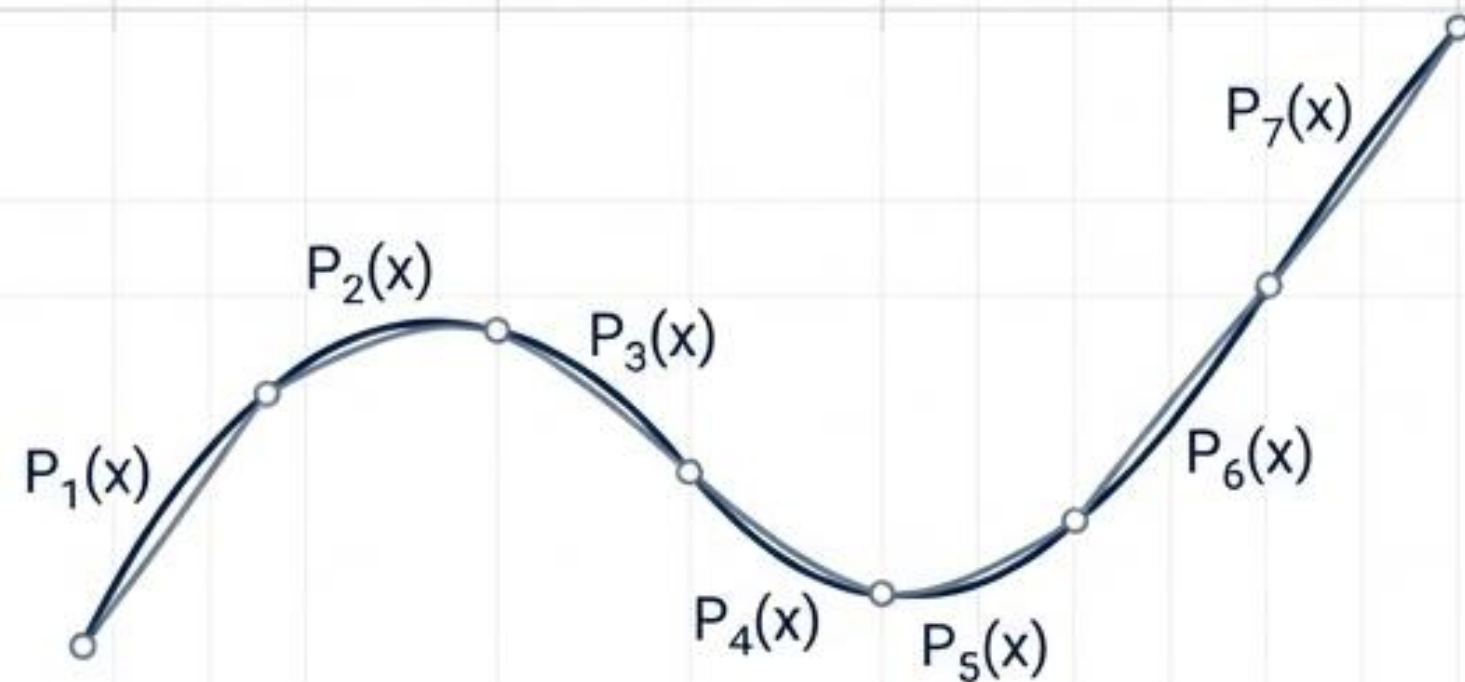
3. Conclusión:

Más información (puntos) distribuida equidistantemente destruye la precisión en los bordes. El polinomio intenta **“bailar” demasiado rápido.**

Evaluación de Soluciones Técnicas

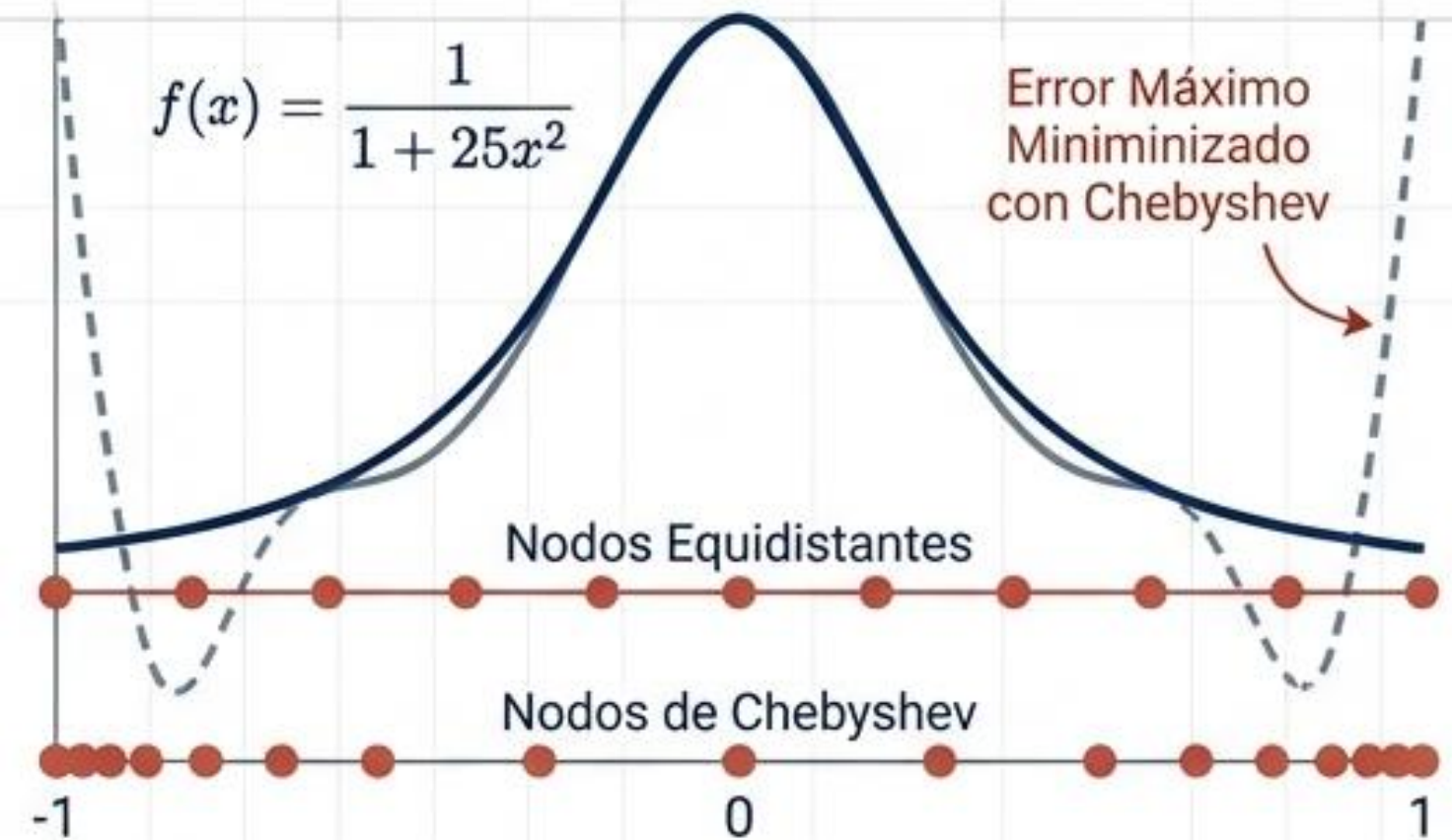
Splines (Trazadores)

Uso de polinomios de bajo grado (cúbicos) por tramos. Elimina la oscilación global.



Nodos de Chebyshev

Redistribución de puntos. Agrupamiento denso en los extremos para minimizar el error máximo.



Referencias Bibliográficas

1. **Burden, R. L., & Faires, J. D.** (2017). *Análisis Numérico*. Cengage Learning.
2. **Chapra, S. C., & Canale, R. P.** (2015). *Métodos Numéricos para Ingenieros*. McGraw-Hill Education.
3. **Sauer, T.** (2018). *Numerical Analysis*. Pearson.
4. **Kiusalaas, J.** (2010). *Numerical Methods in Engineering with MATLAB*. Cambridge University Press.

